

A Grounded Design Memo for a Self-Covering MeTTa Proof Kernel

HOTG, dependent type theory, Megalodon, MeTTa-Pure, MeTTaIL, OSLF, and
MTT/MATT

Prepared for Zar by Oruži

June 2026

Abstract

This memo restates only the grounded, useful content from the discussion about a MeTTa proof kernel capable of supporting both HOTG-style classical higher-order set theory and constructive/dependent type theory. It deliberately separates facts verified from source repositories and papers from design inferences and open hypotheses. The core correction is that Megalodon is not merely an “external oracle”: its repository shows a compact proof checker with proof terms, lambdas, a simple type language, conversion, theorem/proof documents, hash-addressed imports, and multiple object theories. That changes the design center. The right target is a small MeTTa trusted waist that can host multiple proof calculi and theory presentations, while keeping runtime execution, optimization, and general MeTTa rewriting outside the unchecked logical core unless they enter through checked specifications or certificates.

Contents

1	Status and epistemic hygiene	1
2	What the Megalodon repository actually shows	2
2.1	Megalodon is a proof checker with multiple theories	2
2.2	The core proof calculus is small and lambda/proof-term based	2
2.3	Megalodon has a visible web-of-trust layer	3
2.4	The user’s 4CT project is a concrete Megalodon proof artifact	3
3	What the existing MeTTa artifacts already show	3
3.1	PureMeTTa already points to one surface, one core, two honest targets	3
3.2	MeTTaIL and OSLF already connect operational descriptions to type/modal structure	4
4	What Lean, Isabelle/Pure, Dedukti, MMT, MTT, and MATT contribute	4
4.1	Lean is not merely trace verification	4
4.2	Isabelle/Pure, Dedukti, and MMT are framework exemplars	5
4.3	MTT and MATT are mode-theoretic organizers, not magic strength engines	5
5	Does Megalodon change the design? Yes.	5
6	Proposed kernel architecture	6
6.1	Framework core	6
6.2	Object theory layer	6
6.3	Conversion discipline	7
6.4	Imports and web of trust	7

7	What modifications are needed to make MeTTa-Pure support both HOTG and DTT?	8
7.1	Modification 1: make theories first-class	8
7.2	Modification 2: add a simple-type/HOTG profile	8
7.3	Modification 3: add a dependent profile	8
7.4	Modification 4: classify MeTTaIL rewrites	9
7.5	Modification 5: add mode theory without over-axiomatizing it	9
8	Would supporting both make the kernel more pure?	9
9	How this connects to actual programming	9
10	OSLF as execution control, not just verification	10
11	Lean imports, Megalodon hashes, and Greg Meredith’s critique	10
12	Can we prove our kernel is minimal and performant?	11
13	The self-covering coherence picture	11
14	Concrete design rules	12
15	Answer to the central question	12
16	Immediate high-value proof obligations	12
17	References and source grounding	13

1 Status and epistemic hygiene

This document is not a final foundation for MeTTa. It is a grounded design memo. The earlier file `dtc_hotg_metta_brainstorming_cum_heartstorming` is explicitly a brainstorming and heartstorming document, not final project knowledge. It contains useful ideas, wrong turns, and model-generated speculation. Therefore this memo uses three labels.

Grounded fact. Something observed directly in the Megalodon, MeTTaIL, OSLF, or PureMeTTa sources, or in a cited paper/manual.

Design inference. A conclusion that follows reasonably from the facts and project goals, but still requires implementation and proof.

Open obligation. A theorem, engineering proof, benchmark, or artifact that must exist before the inference becomes trusted.

The guiding correction is simple: covering or embedding Lean, Megalodon, Isabelle/Pure, DTT, HOTG, or MATT is not the same thing as outsourcing trust to external oracles. The ideal is self-covering: faithful internal models, checked translations, and commutative coherence diagrams that make our own system more correct.

2 What the Megalodon repository actually shows

2.1 Megalodon is a proof checker with multiple theories

Grounded fact. The repository README says Megalodon is “an open source interactive theorem prover and proof checker” whose main application is Proofgold documents. It supports four Proofgold theories: Egal HOTG, Mizar HOTG, hereditarily finite sets, and an intuitionistic HOAS theory. Egal is the default theory. This is not hearsay; it is in the repository README.[1]

Grounded fact. The installation notes show command-line support for theory selection and proof/document infrastructure: `-hf`, `-hoas`, `-mizar`, `-I` for signature files, `-owned` for ownership files, and `-pfg` for Proofgold output. Signature files are explicitly not allowed to contain proofs.[2]

Design inference. Megalodon is best read as evidence for a small kernel/framework design: one compact checking core, several theories above it, and a web-of-trust/import layer. It should not be dismissed as a mere classical-set-theory oracle. It is a useful proof-kernel exemplar and a covering target.

2.2 The core proof calculus is small and lambda/proof-term based

Grounded fact. Megalodon’s exported syntax interface contains a tiny type language:

`TpVar, Prop, Set, Ar(tp, tp).`

Terms include de Bruijn variables, global term hashes, primitives, type application, term application, lambda, implication, and universal quantification. Proofs include hypotheses, known theorem hashes, type application, term application, proof application, proof lambda, and type lambda.[3]

In abbreviated form, the interface is:

Listing 1: Megalodon core syntax shape, abbreviated from `src/syntax.mli`

```
type tp = TpVar of int | Prop | Set | Ar of tp * tp

type tm =
  | DB of int | TmH of string | Prim of int
  | TpAp of tm * tp | Ap of tm * tm | Lam of tp * tm
  | Imp of tm * tm | All of tp * tm

type pf =
  | Hyp of int | Known of string
  | PTPAp of pf * tp | PTmAp of pf * tm | PPfAp of pf * pf
  | PLam of tm * pf | TLam of tp * pf
```

Grounded fact. The checker extracts and checks term types; application requires a function type, lambda forms an arrow type, implication and universal quantification are checked as propositions. The proof checker handles universal instantiation, implication elimination, proof lambda, and type lambda. Proposition checking reduces to conversion between the extracted proposition and the expected proposition.[4, 6]

Grounded fact. Conversion is based on beta/eta normalization plus controlled delta expansion of definitions. It is not arbitrary MeTTa rewriting.[5]

Design inference. Megalodon’s “C-like” strength comes from a very small, permissive logical kernel plus powerful object theories, libraries, imports, and automation. This supports Zar’s intuition that a MeTTa proof kernel should be made purer by separating the framework from theory-specific commitments. It does *not* imply that the exact Megalodon calculus is sufficient for native dependent type theory.

2.3 Megalodon has a visible web-of-trust layer

Grounded fact. The examples README explains that previous definitions may be referred to opaquely by Merkle roots or transparently by definitions. The `-owned` option supplies an ownership file listing definitions and theorems published in Proofgold. Megalodon warns if an axiom has not yet been proven and errors if a definition hash is not listed in the ownership file.[7]

Grounded fact. The same examples README mentions a translation of Metamath’s `set.mm` into 40,558 Megalodon-checkable theorems and an older Mizar Mathematical Library snapshot in Megalodon format, though without proofs.[7]

Design inference. For MeTTa, imports should be explicit proof-provenance objects: name, content hash, theory, axioms used, checker profile, dependencies, and trust status. Megalodon and Proofgold make that reality visible rather than pretending import trust is invisible.

2.4 The user’s 4CT project is a concrete Megalodon proof artifact

Grounded fact. The `zariuq/ai-agents megalodon/4CT` README states that it contains kernel-verified Megalodon proofs with exit code 0 refuting a specific Goertzel 4CT proof approach. It lists verified files and commands to run Megalodon over them.[8]

Grounded fact. The file `lemma43_refutation.mg` shows ordinary Megalodon proof structure: Variable, Hypothesis, Definition, Theorem, let, assume, prove, apply, exact, and Qed.[9]

Design inference. This is useful because it shows HOTG/set-style proof checking is not an abstract preference. It is already being used in the project to catch real mathematical errors. A future MeTTa kernel should learn from this proof style, import style, and minimal checker discipline.

3 What the existing MeTTa artifacts already show

3.1 PureMeTTa already points to one surface, one core, two honest targets

Grounded fact. `PureMeTTa.tex` says the central claim is that the project should converge to a single elaborated MeTTa-Core compiling to two honest targets: a trusted proof-oriented MeTTa-Pure/PureKernel target and an efficient runtime execution seam backed by MORK/MM2 RuntimeExec.[10]

Grounded fact. The same document compares this to the Lean-style pattern:

surface language $\rightarrow$ elaborated core $\rightarrow$ checked kernel and compiled runtime.

It also emphasizes a closed proof-side semantic waist with canonical normalization, bidirectional checking, parser-free checked canonical evaluation, and preservation.[10]

Grounded fact. The current proof-side branch is described as $\text{MeTTa-Pure} \rightarrow \text{PureKernel} \rightarrow A \rightarrow B \rightarrow C1$, with theorem-level bridge files and a live trusted path through theoremic checking and canonicalization rather than a front-door evaluator.[11]

Design inference. The current MeTTa-Pure direction is already close to the right shape: proof kernel and runtime are connected but not confused. The next refinement is to make the proof kernel more theory-parametric, so that HOTG-like and DTT-like systems can both be covered without hardcoding either one as the only foundation.

3.2 MeTTaIL and OSLF already connect operational descriptions to type/modal structure

Grounded fact. `MeTTaIL.tex` describes MeTTaIL as a Lean 4 framework for pattern-matching rewrite rules with typed premises, a declarative one-step semantics, a proved sound-and-complete executable engine, and automatic derivation of an OSLF behavioral type system for each `LanguageDef`.^[12]

Grounded fact. `TypeSynthesis.lean` states the pipeline explicitly:

$$\text{LanguageDef} \rightarrow \text{RewriteSystem} \rightarrow \text{ReductionSpan} \rightarrow \text{OSLFTypeSystem},$$

with predicates, $\Diamond$, $\Box$, and a Galois connection $\Diamond \dashv \Box$ follows automatically from a generic theorem.^[13]

Grounded fact. The same code defines an executable reduction relation and a declarative reduction relation, and proves the equivalence theorem `langReducesUsing_iff_execUsing`.^[13]

Grounded fact. `TypeSynthesis.lean` defines `langDiamondUsing`, `langBoxUsing`, `langGaloisUsing`, and `langOSLFTUsing`; it then packages a generated `OSLFTypeSystem`.^[14]

Design inference. MeTTaIL is not just a compiler-compiler toy. It is close to a theory/specification language for operational calculi. However, a MeTTaIL rewrite rule is not automatically a kernel conversion rule. Rules must be classified.

Rule class	Logical role	Kernel status
Definitional reduction	Conversion/equality checking	trusted only if terminating/confluent/admitted
Operational step	Runtime/EvalSpec transition	reasoned about propositionally/modal-logically
Optimization rewrite	Compiler/runtime specialization	accepted by certificate/proof
Grounded oracle	FFI/host behavior	admitted only via biform contract

Design inference. Implementing OSLF-like synthesis inside MeTTa/CeTTa can be highly valuable, especially as a precompilation and specialization layer. It should not automatically enlarge the trusted conversion kernel. It should generate obligations, derived types, modalities, and certificates that the kernel can check.

4 What Lean, Isabelle/Pure, Dedukti, MMT, MTT, and MATT contribute

4.1 Lean is not merely trace verification

Grounded fact. Lean’s reference manual says Lean is based on the Calculus of Inductive Constructions with dependent functions, inductive types, a universe hierarchy, and impredicative proof-irrelevant `Prop`. The pure fragment has a computational interpretation with canonical normal forms. Classical axioms such as propositional extensionality, quotient principles, and choice are compatible with code generation partly because proofs and types can be erased, but they can block kernel computation.^[15]

Design inference. Lean’s kernel checking is not merely trace verification. Tactics and automation generate elaborated terms which the kernel checks in a global environment. Trace/certificate checking is one possible proof-producing pattern, but Lean’s main trust story is small-kernel checking of elaborated terms, plus separate compilation/extraction mechanisms. For MeTTa, the ideal is both:

1. ahead-of-time verification of the abstract machine, compiler, and runtime semantics where possible;
2. post-run trace/certificate checking for concrete executions and optimizations.

4.2 Isabelle/Pure, Dedukti, and MMT are framework exemplars

Grounded fact. Isabelle/Pure is the classic example of a generic theorem-prover meta-logic: rules are represented as propositions and proofs are built by combining rules; object logics are formalized in the meta-logic.[16]

Grounded fact. Dedukti’s $\lambda\Pi$ -calculus modulo theory is a logical framework capable of representing many theories, including constructive and classical predicate logic, simple type theory, pure type systems, CIC-like universes, and libraries translated from other systems.[17]

Grounded fact. MMT is a theory/morphism framework in which logics, logical frameworks, and foundations can be represented as theories. It is a benchmark for foundation-independent organization.[18]

Design inference. The MeTTa kernel should not blindly copy Dedukti, Isabelle, MMT, Lean, or Megalodon. They are benchmarks. The question is:

What is the smallest MeTTa waister that supports proof terms, theory specifications, object theories, reductions, imports, mode/morphism structure, runtime certificates, and coherent translations?

4.3 MTT and MATT are mode-theoretic organizers, not magic strength engines

Grounded fact. Multimodal Dependent Type Theory is parameterized by a mode theory containing modes, modalities, and transformations. Its examples include guarded recursion, cohesion, and parametric quantification.[19]

Grounded fact. Shulman’s multimodal adjoint type theory adds adjoint positive/negative modalities and gives semantics in diagrams of toposes and geometric morphisms. This provides an architectural language for modes connected by adjoints.[20]

Design inference. MTT/MATT is not primarily how one gets HOTG’s large set-theoretic strength. Strength comes from axioms, universes, and object theories. MTT/MATT is how one organizes different worlds: proof, program, runtime trace, classical set semantics, constructive computation, capabilities, resources, staging, and evidence state.

5 Does Megalodon change the design? Yes.

The added Megalodon information changes the design center in four ways.

First. The target should not be framed as “DTT kernel plus external HOTG oracle.” Megalodon itself is a compact proof-term/kernel system with multiple theories. A MeTTa proof kernel can aspire to cover this style internally.

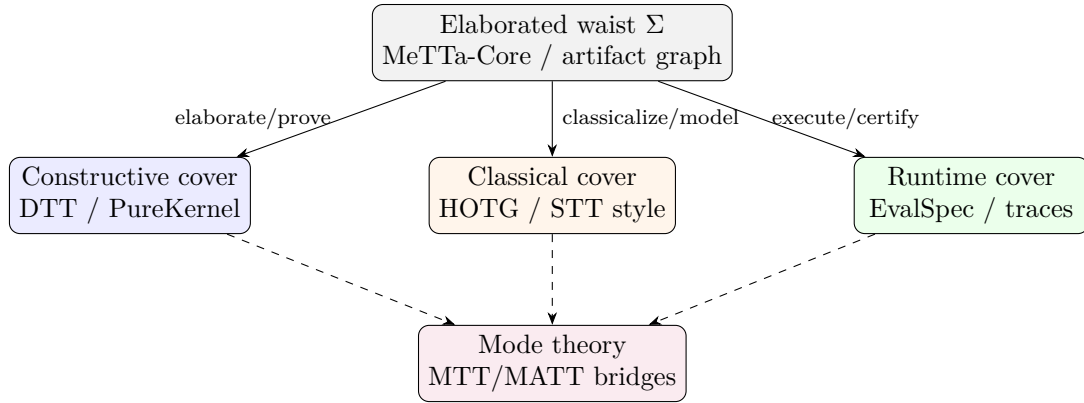
Second. HOTG support is not merely a semantic add-on. It may be one of the cleanest ways to reason about ordinary extensional mathematics, set-like MeTTa semantics, nondeterministic execution relations, large corpora, and classical automation.

Third. The proof kernel should become more parametric and therefore more pure. If HOTG and DTT both have to fit, then the kernel is pressured away from ad hoc Lean imitation and away from ad hoc set-theory primitives. The common waist becomes smaller: binding, terms, proofs, declarations, conversion, imports, theory specifications, and certificates.

Fourth. The hard part shifts from “which one foundation wins?” to “which parts are kernel-common, which are object-theory-specific, and which are mode/morphism structure?”

6 Proposed kernel architecture

Let Σ be the elaborated MeTTa artifact waist. It is not the entire runtime and not one monolithic logic. It is the checked representation where proofs, programs, theories, imports, and certificates can meet.



The trusted kernel should be decomposed into these components.

6.1 Framework core

The framework core should provide:

1. names, de Bruijn or locally nameless binding, alpha-safe substitution, and hashing/content identity;
2. a small type/proposition/proof object language sufficient to express object-theory rules;
3. declarations: parameters, definitions, axioms, theorems, inductive/recursor declarations, and imports;
4. conversion: beta/eta plus admitted definitional reductions, always under a decidability discipline;
5. a proof checker for hypotheses, known theorems, instantiation, implication/application, abstraction, and theory-specific rules;
6. provenance: axioms used, imports used, hashes used, checker version, conversion profile, and trust level.

6.2 Object theory layer

Above the framework core sit theory packages:

HOTG/STT profile. Simple types with **Prop**, **Set**, arrows, universal quantification, implication, equality, membership, set constructors, choice/epsilon, and Tarski-Grothendieck axioms. Megalodon’s **tp**, **tm**, and **pf** syntax is a concrete reference point.

DTT profile. Dependent contexts, Π , Σ , universes, identity/equality, inductives, recursors, iota rules, universe constraints, and a decidable conversion relation for the proof-bearing fragment.

HoTT/cubical profile, optional. Path/equivalence structure, univalence, higher inductives, cubical computation if supported. This is a later profile, not an assumption for the first kernel.

Runtime/EvalSpec profile. Operational semantics of MeTTa programs as relations/traces, not as unconstrained definitional equality.

Modal/mode profile. Modes, modalities, transformations, adjunctions, resource/capability/staging worlds, and bridges between proof/runtime/classical/set worlds.

6.3 Conversion discipline

Conversion is the danger line. For both HOTG and DTT support, conversion should be:

1. decidable in the proof-bearing fragment;
2. small enough to audit;
3. parameterized by a theory profile;
4. closed under checked declarations only;
5. never equal to full MeTTa runtime rewriting unless that fragment is separately proved terminating, confluent, and sound for definitional equality.

6.4 Imports and web of trust

The Megalodon/Proofgold pattern suggests an explicit import object:

Listing 2: Sketch: proof provenance object

```
ProofArtifact := {
  theory      : TheoryId,
  statement   : TermHash,
  proof       : ProofHash,
  checkerProfile : KernelProfileHash,
  conversion  : ConversionProfileHash,
  imports     : List ImportHash,
  axiomsUsed  : List AxiomHash,
  status      : proved | assumed | imported | stale | rejected,
  source      : local | chain | generated | external
}
```

Design inference. Lean-like module imports, Megalodon/Proofgold-style hash imports, MMT-style theory morphisms, and MeTTa atomspace objects should converge here. This would make trust explicit and queryable.

7 What modifications are needed to make MeTTa-Pure support both HOTG and DTT?

7.1 Modification 1: make theories first-class

MeTTa-Pure should not merely have a hardcoded DTT kernel plus ad hoc classical axioms. It should have first-class theory descriptors:

Listing 3: Sketch: theory descriptor

```
Theory := {
  sorts      : SortSpec,
  terms      : TermConstructors,
  proofs     : ProofConstructors,
  constants  : ConstantDeclarations,
  reductions : DefinitionalReductionRules,
  axioms     : AxiomDeclarations,
  inductives : InductiveDeclarations,
  imports    : ImportDeclarations,
  modes     : Optional ModeSignature
}
```

For HOTG, this descriptor can look Megalodon-like. For DTT, it must include dependent binders, universe levels, inductive families, and recursor computation.

7.2 Modification 2: add a simple-type/HOTG profile

A HOTG profile should include something close to Megalodon’s minimal type grammar:

$$A ::= \alpha \mid \text{Prop} \mid \text{Set} \mid A \rightarrow B.$$

Terms need lambda/application, forall/implication propositions, equality, membership, primitive set constructors, and TG axioms. The object theory can be classical and extensional. Its proof checker can remain small.

This profile should not be optimized for numeric computation. Church numerals and set encodings may be acceptable for proof-checking classical mathematics, but they are not ideal for runtime execution. That is a library/tactic/compiler issue for HOTG, not a reason to pollute the core kernel.

7.3 Modification 3: add a dependent profile

A constructive/dependent profile needs:

1. contexts Γ with dependent declarations;
2. Π -types and lambdas with dependent substitution;
3. Σ -types and pairs/projections;
4. universes $\mathcal{U}_0, \mathcal{U}_1, \dots$, with clear predicativity/cumulativity rules;
5. identity/equality types;
6. inductive declarations with generated recursors/eliminators;
7. iota computation rules generated from declarations, not hand-coded per family;
8. subject reduction, decidable conversion, and universe consistency obligations.

7.4 Modification 4: classify MeTTaIL rewrites

MeTTaIL and GSLT-style declarations should feed the kernel, but not by dumping all rewrites into conversion. Each rule must be tagged:

1. definitional rule, admitted to conversion with proof of termination/confluence or a restricted checker;
2. propositional operational rule, available for EvalSpec theorems and OSLF modalities;
3. optimization rule, available to the compiler only with a certificate;
4. oracle/grounded rule, available only through a biform logical contract.

7.5 Modification 5: add mode theory without over-axiomatizing it

The MTT/MATT layer should make MeTTa modes mathematical rather than merely pragmatic flags. Likely modes include:

Program	RuntimeTrace	DTTProof	HOTGProof
Classical	Resource	Capability	Oracle
EvidenceState	Staging	Grounded	Optimization

Mode morphisms include erase, quote, reflect, certify, classicalize, model, execute, specialize, forget, and approximate. Some are adjoints. Some are merely translations or certificate checkers. The kernel should not pretend all bridges are the same kind of map.

8 Would supporting both make the kernel more pure?

Yes, if done correctly. The requirement to support both HOTG and DTT forces theory-specific assumptions out of the core. The common kernel becomes more like:

A small framework for checking terms, proofs, conversions, declarations, imports, and theory-specific rule packages.

That is purer than a kernel that hardcodes only Lean-like inductives or only Megalodon-like HOTG primitives.

No, if done incorrectly. If supporting both means one undifferentiated soup of classical axioms, dependent reductions, set-theoretic membership, runtime effects, nondeterministic rewriting, and compiler optimizations inside definitional equality, then it becomes less pure and less trustworthy.

The rule is:

Common framework in the trusted waist; object-theory axioms and reductions in explicit profiles; optimizations outside the kernel unless certificate-checked.

9 How this connects to actual programming

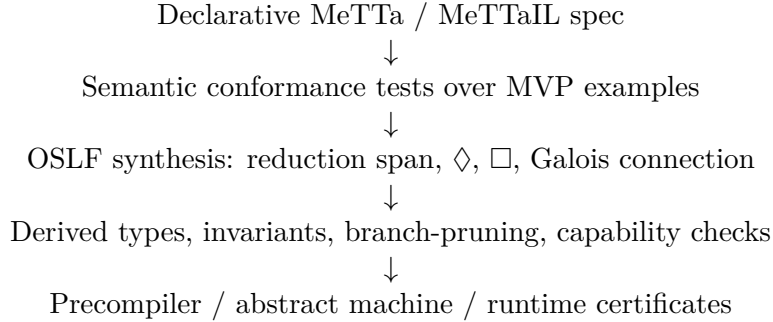
This is not only type safety and software verification, although those are already valuable. A self-covering MeTTa kernel can power up programming in at least nine ways.

1. **Type-directed execution.** Dependent/capability types can prune impossible branches before runtime search.

2. **Mode-directed execution.** A pragma can become a mathematical mode annotation: execute, prove, quote, certify, stage, resource-bound, classical, constructive, grounded.
3. **Proof-carrying optimization.** Fast paths and streaming optimizations can be accepted because they carry refinement certificates.
4. **Runtime trace certification.** Executions can emit traces checked against EvalSpec.
5. **Verified-ahead abstract machine.** The MeTTa abstract machine and compiler passes can be proved correct before running, not only checked afterward.
6. **Security by non-inhabitation.** Capability failures become negative theorems: no proof of authority, no typed execution path.
7. **Classical semantic reasoning.** HOTG can reason naturally about sets of states, nondeterministic relations, extensional models, and large mathematical corpora.
8. **Constructive runtime guidance.** DTT can provide proof-relevant, executable witnesses and safe dependent interfaces.
9. **Self-analysis.** Programs, traces, optimizers, and theorem importers can be quoted back into Σ and reasoned about by the system itself.

10 OSLF as execution control, not just verification

The most promising pipeline is:



Grounded fact. The current Lean OSLF code already derives $\diamond$, $\square$, a Galois connection, and `OSLFTypeSystem` from `LanguageDef`.[\[13, 14\]](#)

Design inference. Therefore, a native OSLF or OSLF-inspired implementation in CeTTa could improve runtime performance at the precompilation/specialization layer. It can derive admissible fast paths, branch filters, and runtime obligations. The kernel then checks the small certificates rather than trusting the optimizer.

11 Lean imports, Megalodon hashes, and Greg Meredith’s critique

Greg Meredith’s critique, as summarized in the discussion, is roughly that Lean lives in one large top-down universe, while a more relational/foundation-independent system should reason about many theories and their morphisms. This is partially correct.

Lean is extremely effective: a small CIC kernel plus a global environment, powerful elaborator, tactics, and mathlib. It can encode alternative theories, and it distinguishes computational fragments from classical/proof-irrelevant fragments. But its kernel architecture is not primarily a theory-graph/morphism system.

Megalodon/Proofgold makes a different tradeoff: names and imports are content-addressed, previous facts can be referenced by Merkle roots, and ownership/provenance are user-visible.[7]

MMT and MeTTaIL/GSLT-style theory descriptions push even further toward explicit theories, morphisms, syntax, rules, and generated semantics. That seems closer to the MeTTa dream: many calculi, many modes, many execution regimes, one self-covering substrate.

12 Can we prove our kernel is minimal and performant?

Open obligation. An absolute theorem saying “this is the minimal performant kernel satisfying all true future criteria” is not a realistic formal target. Minimality and performance depend on encodings, hardware, workload distributions, library engineering, compiler choices, and future criteria.

Design inference. But we can prove useful relative theorems:

1. **Admission minimality.** Every trusted primitive has a corpus obligation and cannot be expressed in the already-admitted fragment without violating performance or soundness criteria.
2. **Coverage minimality.** A smaller profile fails to cover a named set of examples, translations, or proofs.
3. **Conservativity.** Adding a theory package does not change existing judgments outside its marked interface.
4. **Adequacy.** The encoding of HOTG, DTT, or EvalSpec represents exactly the intended derivations up to a stated equivalence.
5. **Bridge coherence.** Translations around a square commute on the marked overlap.
6. **Complexity bounds.** Particular checking, conversion, or specialization algorithms satisfy formal resource bounds on chosen fragments.

This is enough to make kernel design mathematically disciplined without pretending to solve an impossible universal optimization theorem.

13 The self-covering coherence picture

The ideal can be stated precisely.

Let Σ be the elaborated MeTTa waist. Let R be runtime semantics, D constructive/dependent semantics, H HOTG/set semantics, and S HOL/STT proof-engineering semantics. A term, trace, theorem, or optimization is in the trustworthy overlap when its images commute under the relevant maps:

$$\begin{aligned}
&\text{cert}_D \circ \text{run} \simeq \text{stepSpec}_D \circ \text{prove}, \\
&\text{cert}_H \circ \text{run} \simeq \text{relSem}_H \circ \text{set}, \\
&\text{model}_{D \rightarrow H} \circ \text{prove} \simeq \text{set}, \\
&\text{class}_{D \rightarrow S} \circ \text{prove} \simeq \text{hol}, \\
&\text{quote} \circ \text{run} \simeq \text{id}_\Sigma \quad \text{on traceable fragments.}
\end{aligned}$$

MATT is useful when these maps are better understood as modes, modalities, and adjunctions. It should organize bridges, not erase distinctions between object theories.

14 Concrete design rules

1. Do not treat brainstorming documents as authority. Treat source code, checked theorems, and cited papers as authority.
2. Do not call Megalodon merely an external oracle. It is a compact proof checker and a reference kernel style for HOTG/STT-like covers.
3. Do not identify MeTTaIL rewrites with kernel conversion. Classify rules.
4. Do not optimize inside the proof kernel. Optimize by generating certificates or by proving compiler passes correct.
5. Do not choose HOTG versus DTT prematurely. Use both as pressure tests.
6. Do not make MATT the source of set-theoretic strength. Use it to organize modes and adjoints.
7. Do make imports, axioms, hashes, and dependencies first-class.
8. Do make negative examples first-class: bad capability use, bad index, unsound FFI equality, nonterminating conversion, forged theorem imports.
9. Do use OSLF and Native Type Theory ideas to turn operational semantics into behavioral types and execution control.

15 Answer to the central question

Zar’s hunch is basically right:

Aiming to support both HOTG-like and DTT-like systems can make MeTTa-Pure more pure, if the result is a smaller framework kernel plus explicit object-theory profiles, not a larger blended logic.

Megalodon shows that a tiny proof-term kernel can support classical higher-order set theories and real proof corpora. Lean/Coq/Agda show that dependent type theory is invaluable for constructive proofs, programs, and runtime guidance. MeTTaIL/OSLF show that operational semantics can generate modal/behavioral type structure. MTT/MATT show how mode-indexed bridges can be made formal.

The theoretical ideal, to the extent currently visible, is therefore:

A self-covering MeTTa substrate with one small trusted checking waist, explicit theory descriptions, theory/mode morphisms, HOTG/STT and DTT profiles, OSLF-derived operational modalities, proof provenance hashes, and runtime certificates. Runtime optimization lives outside the kernel but is made trustworthy by checked specifications, certificates, and bridge theorems.

16 Immediate high-value proof obligations

1. Encode a tiny Megalodon-like simple-type/HOTG kernel fragment in MeTTa-Pure or Lean and prove its checker sound.
2. Encode a tiny DTT fragment with Π , universes, and one inductive family; prove preservation and decidable conversion.

3. Define a common declaration/provenance format that can represent Megalodon-style hashes and Lean-style checked declarations.
4. Classify MeTTaIL rules into definitional, operational, optimization, and oracle classes.
5. Use OSLF on a small MeTTa EvalSpec fragment and derive $\Diamond$, $\Box$, and a branch-pruning invariant.
6. Prove one commuting square: the same tiny MeTTa transition relation represented as DTT inductive relation and HOTG set relation agrees on finite examples.
7. Prove one negative capability theorem: no inhabitant of the authority type means no executable read path.

17 References and source grounding

References

- [1] Megalodon repository, README. Lines inspected: 5–24. States that Megalodon is an open source interactive theorem prover and proof checker; lists Egal HOTG, Mizar HOTG, HF, and HOAS; Egal is default. <https://github.com/ai4reason/Megalodon>
- [2] Megalodon repository, INSTALL. Lines inspected: 3–21. Documents OCaml build and command-line options including `-I`, `-owned`, `-pfg`, `-hf`, `-hoas`, and `-mizar`. <https://github.com/ai4reason/Megalodon>
- [3] Megalodon repository, `src/syntax.mli`. Lines inspected: 23–52. Defines simple types, term syntax, and proof syntax. <https://github.com/ai4reason/Megalodon/blob/master/src/syntax.mli>
- [4] Megalodon repository, `src/syntax.ml`. Lines inspected: around 1090–1160. Implements term type extraction/checking for application, lambda, implication, and universal quantification. <https://github.com/ai4reason/Megalodon/blob/master/src/syntax.ml>
- [5] Megalodon repository, `src/syntax.ml`. Lines inspected: around 1300–1405. Implements beta/eta conversion with controlled delta expansion. <https://github.com/ai4reason/Megalodon/blob/master/src/syntax.ml>
- [6] Megalodon repository, `src/syntax.ml`. Lines inspected: around 1470–1553. Implements proof application to universal propositions and implications, proof lambdas, and proposition checking by conversion. <https://github.com/ai4reason/Megalodon/blob/master/src/syntax.ml>
- [7] Megalodon repository, `examples/README`. Lines inspected: 19–36, 38–52, 116–128, 147–152, 191–214. Documents `set.mm` translation, MML snapshot, HOTG examples, Merkle roots, ownership files, preambles, and ATP export. <https://github.com/ai4reason/Megalodon/blob/master/examples/README>
- [8] `zariuq/ai-agents/megalodon/4CT/README.md`. Lines inspected: 3–16, 35–50, 83–96. Documents kernel-verified Megalodon 4CT-refutation project and commands. <https://github.com/zariuq/ai-agents/tree/main/megalodon/4CT>
- [9] `zariuq/ai-agents/megalodon/4CT/lemma43_refutation.mg`. Lines inspected: 3–118. Concrete Megalodon proof file. https://github.com/zariuq/ai-agents/blob/main/megalodon/4CT/lemma43_refutation.mg

- [10] zariuq/ai-agents/lean-projects/mettapedia/papers/PureMeTTa.tex. Lines inspected: 99–116 and 125–143. States one surface MeTTa to one elaborated core to proof and runtime targets. <https://github.com/zariuq/ai-agents/blob/main/lean-projects/mettapedia/papers/PureMeTTa.tex>
- [11] Same PureMeTTa.tex. Lines inspected: around 210–360. Describes current proof branch, theoremic checking/canonicalization boundary, and semantic waist. <https://github.com/zariuq/ai-agents/blob/main/lean-projects/mettapedia/papers/PureMeTTa.tex>
- [12] zariuq/ai-agents/lean-projects/mettapedia/papers/MeTTaIL.tex. Lines inspected: 37–46, 51–66, 70–140. Describes MeTTaIL syntax, typed premises, declarative semantics, executable engine, OSLF derivation, and 0-sorry coverage table. <https://github.com/zariuq/ai-agents/blob/main/lean-projects/mettapedia/papers/MeTTaIL.tex>
- [13] zariuq/ai-agents/lean-projects/mettapedia/Mettapedia/OSLF/Framework/TypeSynthesis.lean. Lines inspected: 10–39 and 54–90. Documents LanguageDef to OSLF pipeline and executable/declarative bridge. <https://github.com/zariuq/ai-agents/blob/main/lean-projects/mettapedia/Mettapedia/OSLF/Framework/TypeSynthesis.lean>
- [14] Same TypeSynthesis.lean. Lines inspected: 174–208 and 230–314. Defines langDiamondUsing, langBoxUsing, langGaloisUsing, langOSLFUsing, and native types. <https://github.com/zariuq/ai-agents/blob/main/lean-projects/mettapedia/Mettapedia/OSLF/Framework/TypeSynthesis.lean>
- [15] Lean reference manual, “Axioms and Computation.” Documents CIC basis, proof-irrelevant Prop, classical axioms, normalization/canonicity caveats, and erasure for compiled code. <https://lean-lang.org/doc/reference/latest/The-Theory-of-Lean/Axioms-and-Computation/>
- [16] Lawrence C. Paulson. “The Foundation of a Generic Theorem Prover.” Describes Isabelle as a generic theorem prover and meta-logic/logical framework for object logics. <https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-130.pdf>
- [17] Ali Assaf et al. “Dedukti: a Logical Framework based on the $\lambda\Pi$ -Calculus Modulo Theory.” arXiv:2311.07185. <https://arxiv.org/abs/2311.07185>
- [18] Florian Rabe and Michael Kohlhase. MMT literature on scalable/foundation-independent module and theory systems. <https://uniformal.github.io/>
- [19] Daniel Gratzer, G. A. Kavvos, Andreas Nuyts, and Lars Birkedal. “Multimodal Dependent Type Theory.” arXiv:2011.15021. <https://arxiv.org/abs/2011.15021>
- [20] Michael Shulman. “Semantics of multimodal adjoint type theory.” arXiv:2303.02572. <https://arxiv.org/abs/2303.02572>
- [21] Christian Williams and Mike Stay. “Native Type Theory.” arXiv:2102.04672. <https://arxiv.org/abs/2102.04672>
- [22] Chad E. Brown and Karol Pak. “A Tale of Two Set Theories.” CICM 2019 / arXiv:1907.08368. <https://arxiv.org/abs/1907.08368>